

■ C++ DSA Engineering Reference & Study Notes

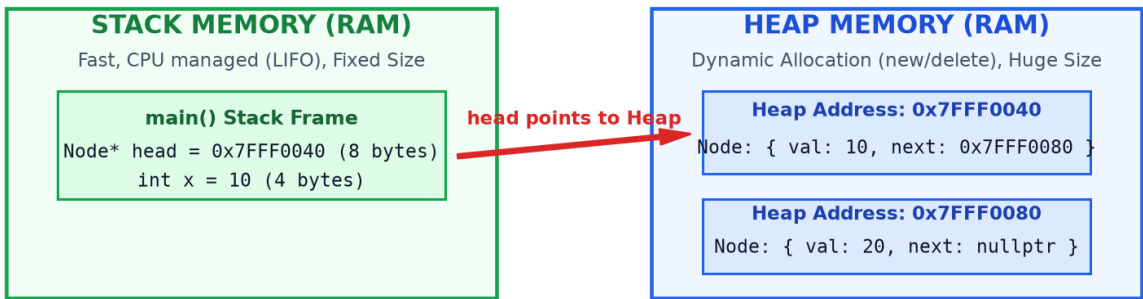
Author: Amit | Complete First-Principles Textbook: Memory, Pointers, Linked Lists & Algorithms

Module 1: C++ Hardware Memory Model & Pointer Mechanics

When your C++ program runs on a modern operating system (Windows/Linux x86_64), the OS allocates a virtual address space split into distinct hardware zones:

- 1. **Code / Text Segment:** Compiled machine instructions (read-only binary).
- 2. **Data Segment:** Global and static variables initialized at program boot.
- 3. **Stack Memory (LIFO):** High-speed memory allocated automatically for function call frames, local variables, and return addresses. Managed directly by the CPU Stack Pointer (SP) register. Stack overflow occurs when recursion runs unbounded.
- 4. **Heap Memory:** Dynamic memory managed at runtime via new and delete (or malloc/free in C). Requires manual lifetime management. Failing to delete heap-allocated nodes leads to memory leaks!

C++ Hardware Memory Model: Stack vs Heap Architecture



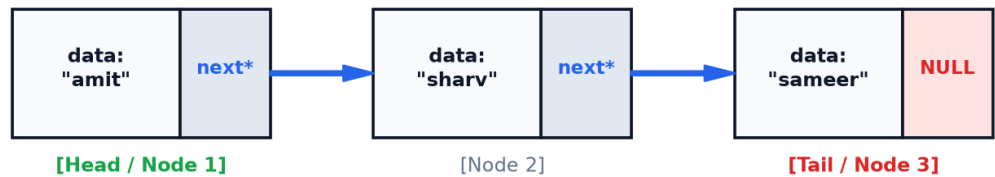
Pointer Operators Deep Dive: Hardware Meaning

Syntax	Operator Name	Hardware / Assembly Action	Example Usage
&x;	Address-of	Fetches the physical RAM address (64-bit integer) where variable x lives.	int* ptr = &x;
*ptr	Dereference	Tells CPU to jump to the address inside ptr and read/write the value stored there.	*ptr = 25;
ptr->val	Arrow Operator	Shorthand for (*ptr).val. Dereferences the pointer first, then accesses object field.	head->name;
int** dptr	Double Pointer	Stores the address of another pointer variable. Used when modifying head pointer inside functions.	int** dptr = &ptr;

Module 2: Linked List Architecture & Dynamic Traversal (Phase 4)

Unlike contiguous memory structures (like std::vector or int arr[]), a **Linked List** is a non-contiguous data structure where elements (called **Nodes**) are dynamically allocated individually on the Heap and linked via pointers.

Singly Linked List: Pointer Chaining & Dynamic Traversal



```
// 1. Fundamental Node Class Definition
class Node {
public:
    string name;
    Node* next;
    Node(string val) : name(val), next(nullptr) {}
};

// 2. Dynamic Traversal (While Loop vs Hardcoding head->next->next)
void printList(Node* head) {
    Node* temp = head; // Start temp pointer at head
    while (temp != nullptr) {
        cout << temp->name << " -> ";
        temp = temp->next; // Advance to next node in RAM
    }
    cout << "NULL\n";
}

// 3. Insert at Head (O(1) Constant Time)
void insertAtHead(Node* &head, string val) {
    Node* newNode = new Node(val);
    newNode->next = head; // Step 1: Connect newNode to current head
    head = newNode;      // Step 2: Move head to point to newNode
}
```

Critical Edge-Cases in Linked Lists (Interview Traps)

Scenario	Condition	Required Defensive Code	Risk if Ignored
Empty List	head == nullptr	Check if(!head) return; before accessing head->next.	Segmentation Fault (Crash)
Single Node	head->next == nullptr	Handle special deletion/reversal logic when size is exactly 1.	Dangling pointer or memory leak
Delete Last Node	temp->next == tail	Stop traversal at second-last node, set prev->next = nullptr.	Reading freed memory (UB)
Odd/Even Length	Fast/Slow pointers	While loop condition: fast != nullptr && fast->next != nullptr.	Null pointer dereference at tail

Module 3: Classic Algorithms Complexity & Mechanics Reference

Binary Search	$O(1)$ / $O(\log N)$ / $O(\log N)$	$O(1)$	Search space reduction by 50% each iteration: $\text{mid} = \text{st} + (\text{end}-\text{st})/2$.
Merge Sort	$O(N \log N)$ all cases	$O(N)$	Divide & conquer: Recursively divide in half, merge 2 sorted sub-arrays.
Kadane's Algorithm	$O(N)$ single pass	$O(1)$	Discard negative prefix sum: if($\text{curr_sum} < 0$) $\text{curr_sum} = 0$;
Moore's Voting	$O(N)$ single pass	$O(1)$	Frequency balance: Cancel non-matching elements against majority candidate.
Two Pointers (Sorted)	$O(N)$ linear scan	$O(1)$	Converging left & right pointers based on $\text{sum} < \text{target}$ or $\text{sum} > \text{target}$.
Monotonic Stack	$O(N)$ amortized	$O(N)$	Maintain decreasing/increasing stack to find Next Greater / Stock Span in $O(1)$ amortized.